Consistent Hashing (New)

Let's learn about Consistent Hashing and its usage.

Background

While designing a scalable system, the most important aspect is defining how the data will be partitioned and replicated across servers. Let's first define these terms before moving on:

Data partitioning: It is the process of distributing data across a set of servers. It improves the scalability and performance of the system.

Data replication: It is the process of making multiple copies of data and storing them on different servers. It improves the availability and durability of the data across the system.

Data partition and replication strategies lie at the core of any distributed system. A carefully designed scheme for partitioning and replicating the data **enhances the performance, availability, and reliability of the system** and also defines how efficiently the system will be scaled and managed.

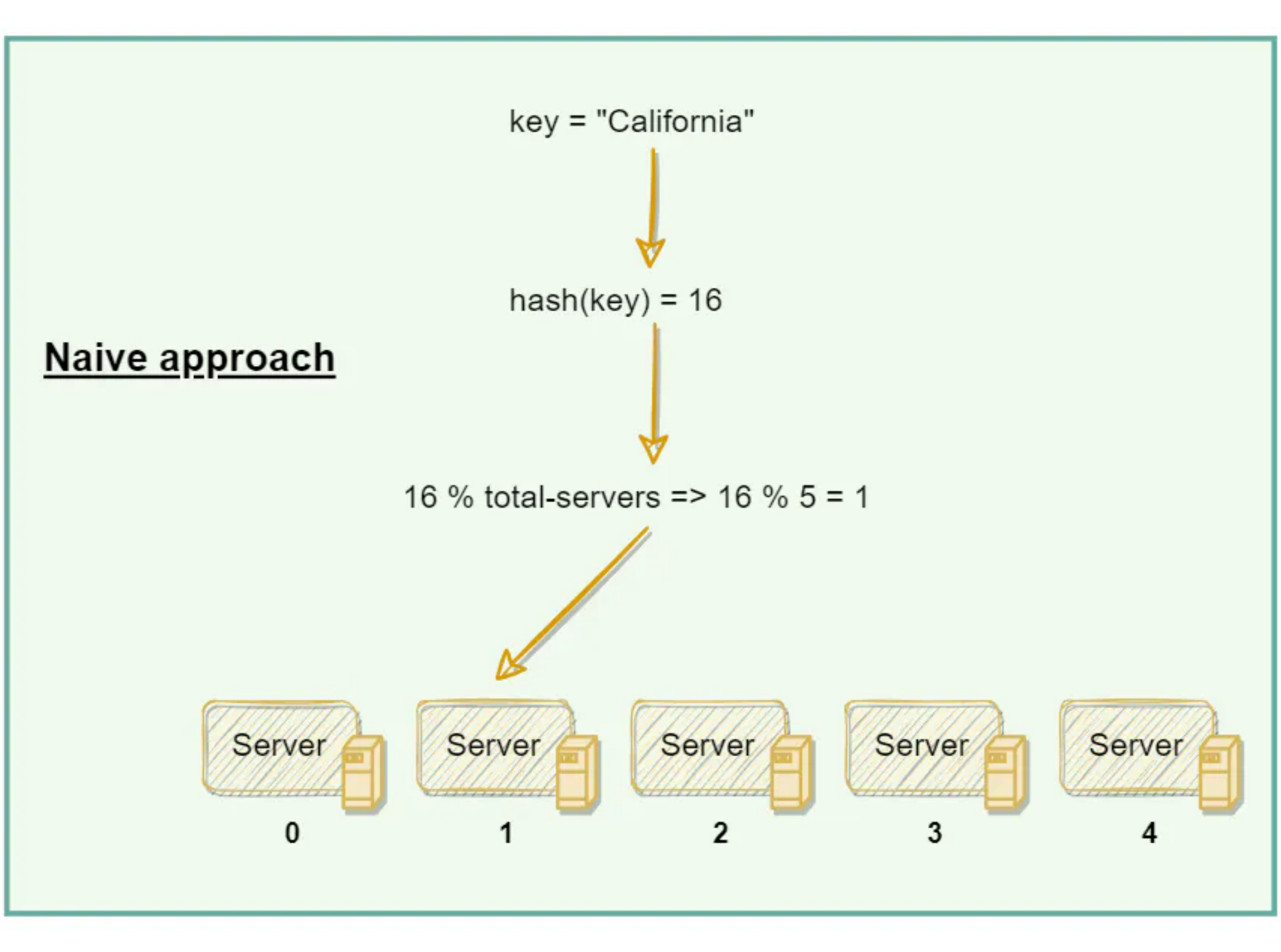
David Karger et al. first introduced Consistent Hashing in their 1997 paper and suggested its use in distributed caching. Later, Consistent Hashing was adopted and enhanced to be used across many distributed systems. In this lesson we will see how Consistent Hashing efficiently solves the problem of data partitioning and replication.

What is data partitioning?

As stated above, the act of distributing data across a set of nodes is called data partitioning. There are two challenges when we try to distribute data:

1. How do we know on which node a particular piece of data will be stored?
2. When we add or remove nodes, how do we know what data will be moved from existing nodes to the new nodes? Additionally, how can we minimize data movement when nodes join or leave?

A naive approach will use a suitable hash function to map the data key to a number. Then, find the server by applying modulo on this number and the total number of servers. For example:



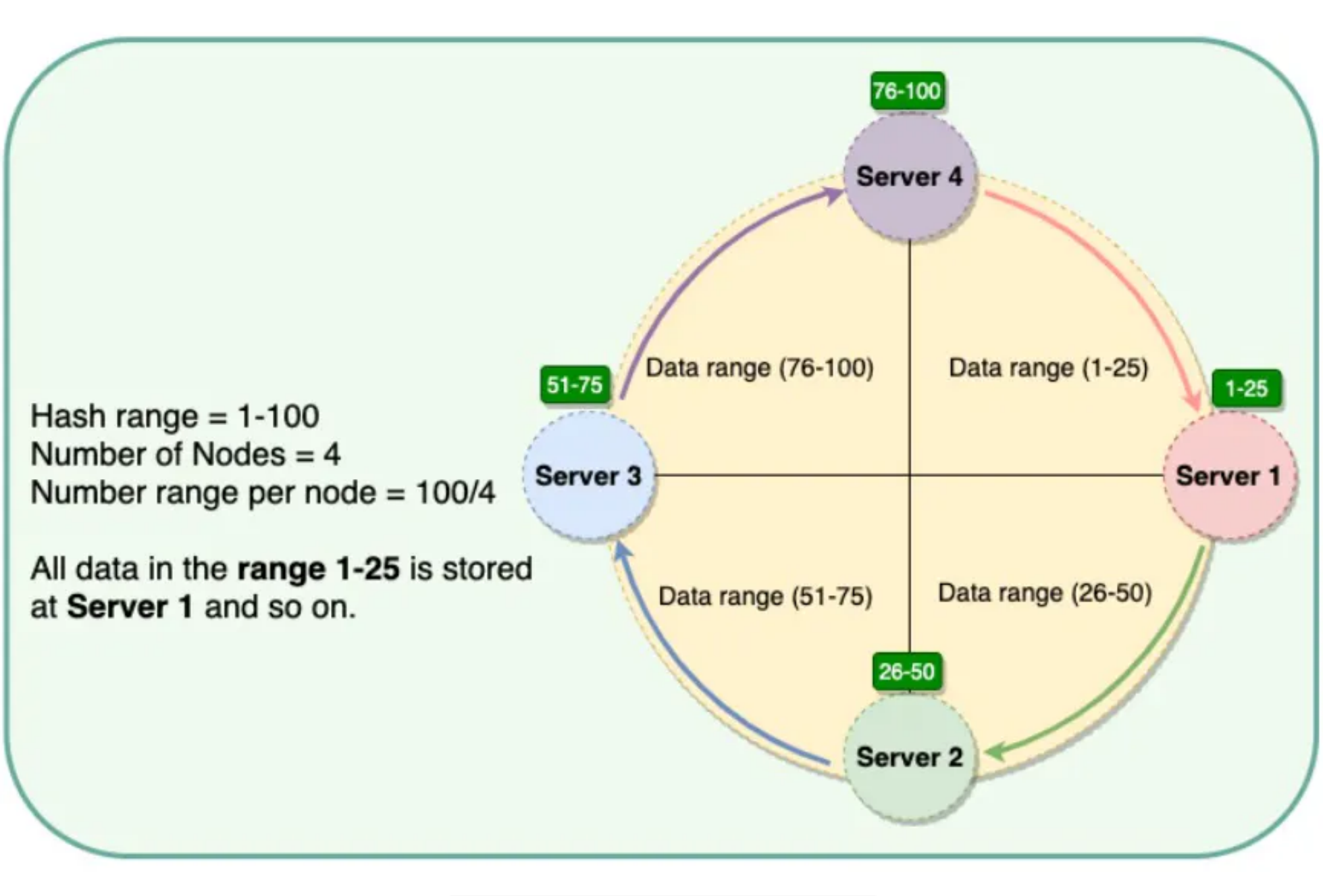
Data partitioning using simple hashing

The scheme described in the above diagram solves the problem of finding a server for storing/retrieving the data. But when we add or remove a server, all our existing mappings will be broken. This is because the total number of servers will be changed, which was used to find the actual server storing the data. So to get things working again, we have to **remap all the keys** and move our data based on the new server count, which will be a **complete mess**!

Consistent Hashing to the rescue

Distributed systems can use Consistent Hashing to distribute data across nodes. Consistent Hashing maps data to physical nodes and ensures that **only a small set of keys move when servers are added or removed.**

Consistent Hashing stores the data managed by a distributed system in a ring. Each node in the ring is assigned a range of data. Here is an example of the consistent hash ring:

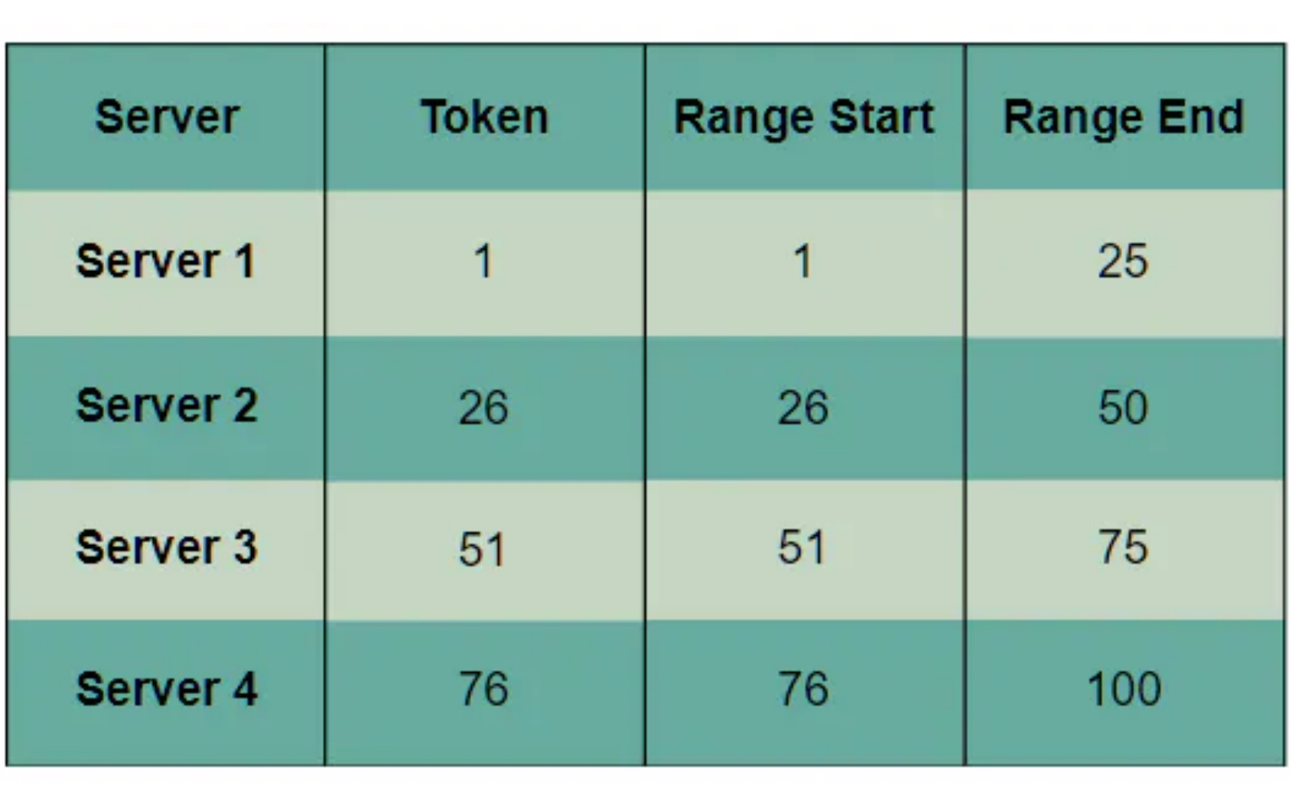


Consistent Hashing ring

With consistent hashing, the ring is divided into smaller, predefined ranges. Each node is assigned one of these ranges. The start of the range is called a **token.** This means that each node will be assigned one token. The range assigned to each node is computed as follows:

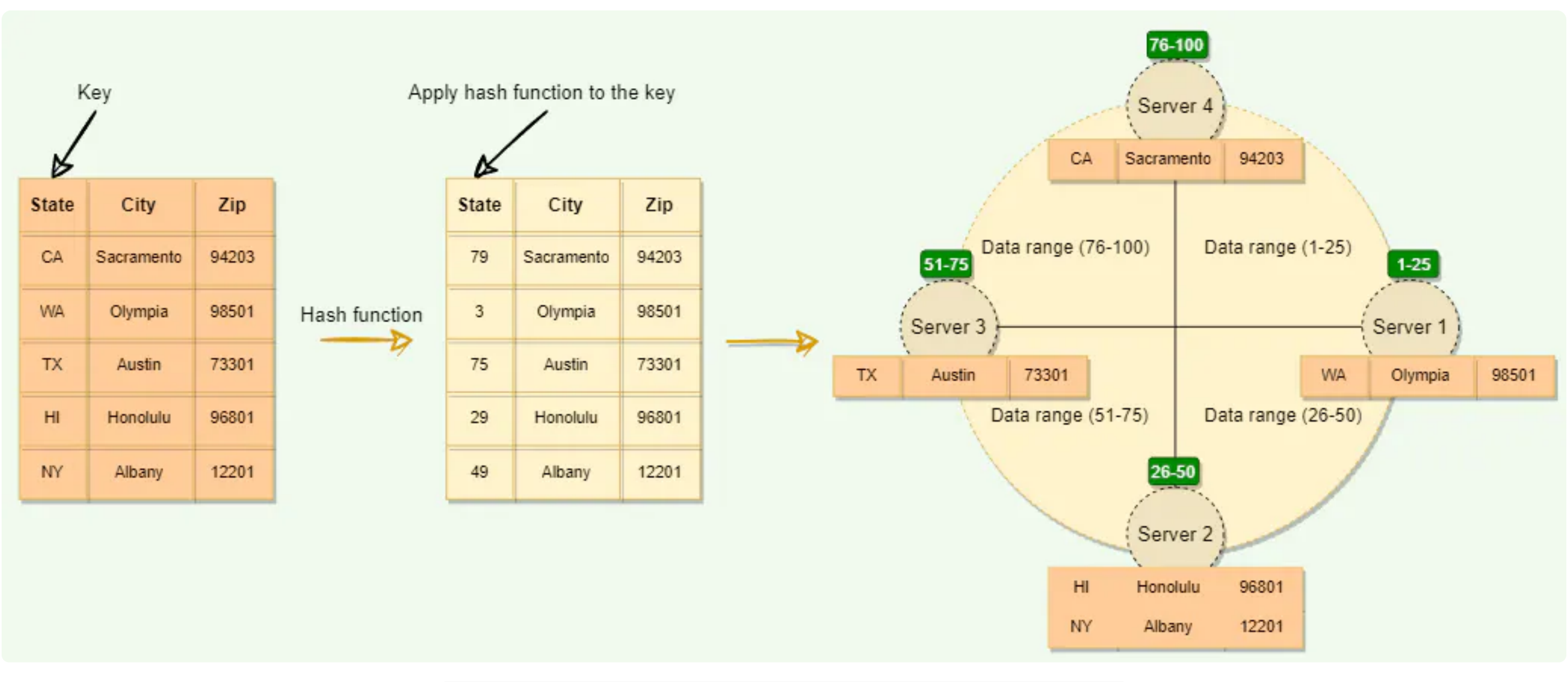
Range start: Token value
Range end: Next token value - 1

Here are the tokens and data ranges of the four nodes described in the above diagram:



Server	Token	Range Start	Range End
Server 1	1	1	25
Server 2	26	26	50
Server 3	51	51	75
Server 4	76	76	100

Whenever the system needs to read or write data, the first step it performs is to apply the MD5 hashing algorithm to the key. The output of this hashing algorithm determines within which range the data lies and hence, on which node the data will be stored. As we saw above, each node is supposed to store data for a fixed range. Thus, the hash generated from the key tells us the node where the data will be stored.



State	City	Zip
CA	Sacramento	94203
WA	Olympia	98501
TX	Austin	73301
HI	Honolulu	96801
NY	Albany	12201

State	City	Zip
79	Sacramento	94203
3	Olympia	98501
75	Austin	73301
29	Honolulu	96801
49	Albany	12201

CA	Sacramento	94203

TX	Austin	73301

WA	Olympia	98501

HI	Honolulu	96801
NY	Albany	12201

Distributing data on the Consistent Hashing ring

The Consistent Hashing scheme described above works great when a node is added or removed from the ring, as in these cases, since only the next node is affected. For example, when a node is removed, the next node becomes responsible for all of the keys stored on the outgoing node. However, this scheme can **result in non-uniform data and load distribution**. This problem can be solved with the help of Virtual nodes.

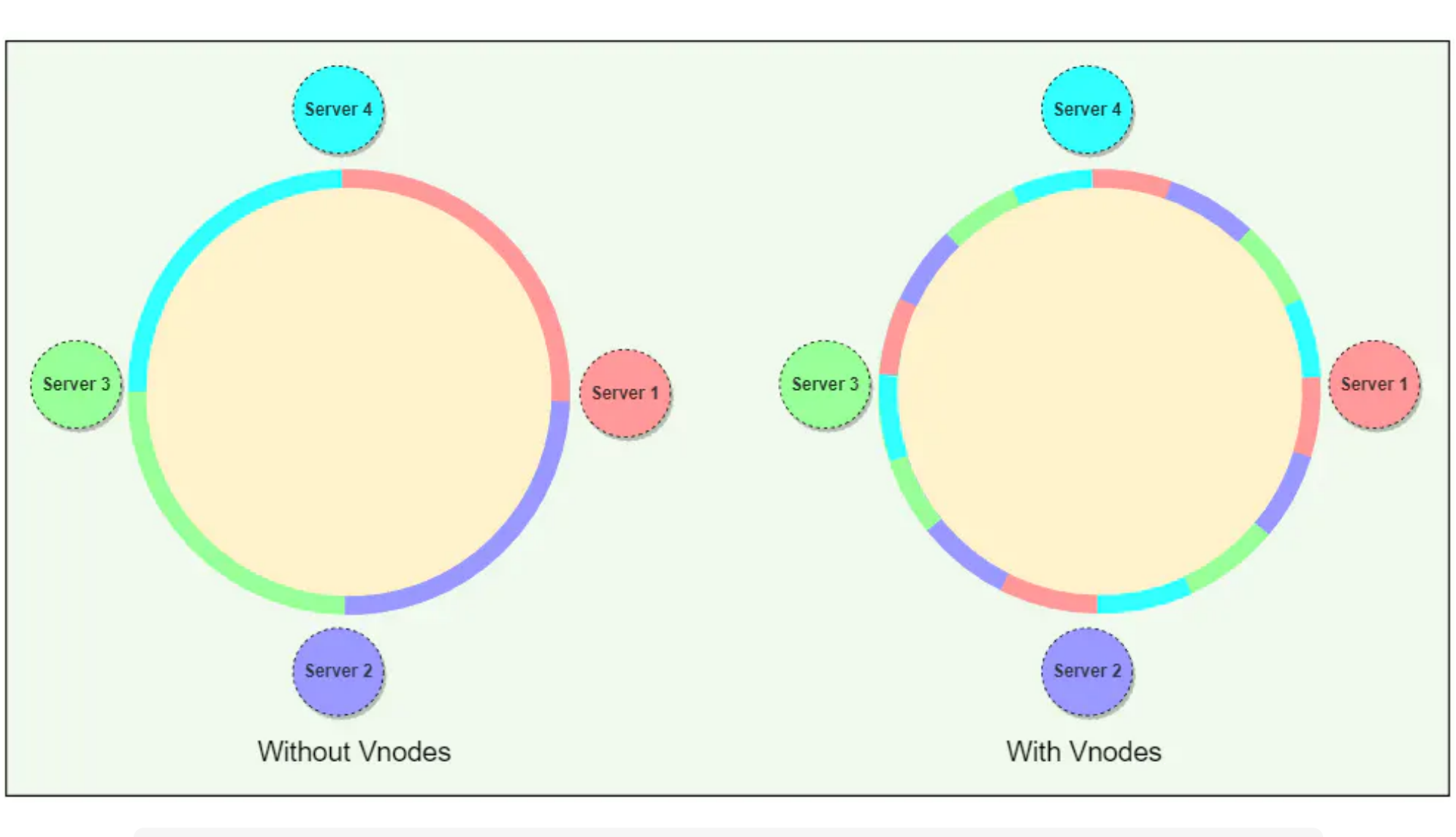
Virtual nodes

Adding and removing nodes in any distributed system is quite common. Existing nodes can die and may need to be decommissioned. Similarly, new nodes may be added to an existing cluster to meet growing demands. To efficiently handle these scenarios, Consistent Hashing makes use of virtual nodes (or Vnodes).

As we saw above, the basic Consistent Hashing algorithm assigns a single token (or a consecutive hash range) to each physical node. This was a static division of ranges that requires calculating tokens based on a given number of nodes. This scheme made adding or replacing a node an expensive operation, as, in this case, we would like to rebalance and distribute the data to all other nodes, resulting in moving a lot of data. Here are a few potential issues associated with a manual and fixed division of the ranges:

- **Adding or removing nodes:** Adding or removing nodes will result in recomputing the tokens causing a significant administrative overhead for a large cluster.
- **Hotspots:** Since each node is assigned one large range, if the data is not evenly distributed, some nodes can become hotspots.
- **Node rebuilding:** Since each node's data might be replicated (for fault-tolerance) on a fixed number of other nodes, when we need to rebuild a node, only its replica nodes can provide the data. This puts a lot of pressure on the replica nodes and can lead to service degradation.

To handle these issues, Consistent Hashing introduces a new scheme of distributing the tokens to physical nodes. Instead of assigning a single token to a node, the hash range is divided into multiple smaller ranges, and each physical node is assigned several of these smaller ranges. Each of these subranges is considered a Vnode. With Vnodes, instead of a node being responsible for just one token, it is responsible for many tokens (or subranges).



Mapping Vnodes to physical nodes on a Consistent Hashing ring

Advantages of Vnodes

Vnodes gives the following advantages:

1. As Vnodes help spread the load more evenly across the physical nodes on the cluster by dividing the hash ranges into smaller subranges, this speeds up the rebalancing process after adding or removing nodes. When a new node is added, it receives many Vnodes from the existing nodes to maintain a balanced cluster. Similarly, when a node needs to be rebuilt, instead of getting data from a fixed number of replicas, many nodes participate in the rebuild process.
2. Vnodes make it easier to maintain a cluster containing heterogeneous machines. This means, with Vnodes, we can assign a high number of sub-ranges to a powerful server and a lower number of sub-ranges to a less powerful server.
3. In contrast to one big range, since Vnodes help assign smaller ranges to each physical node, this decreases the probability of hotspots.

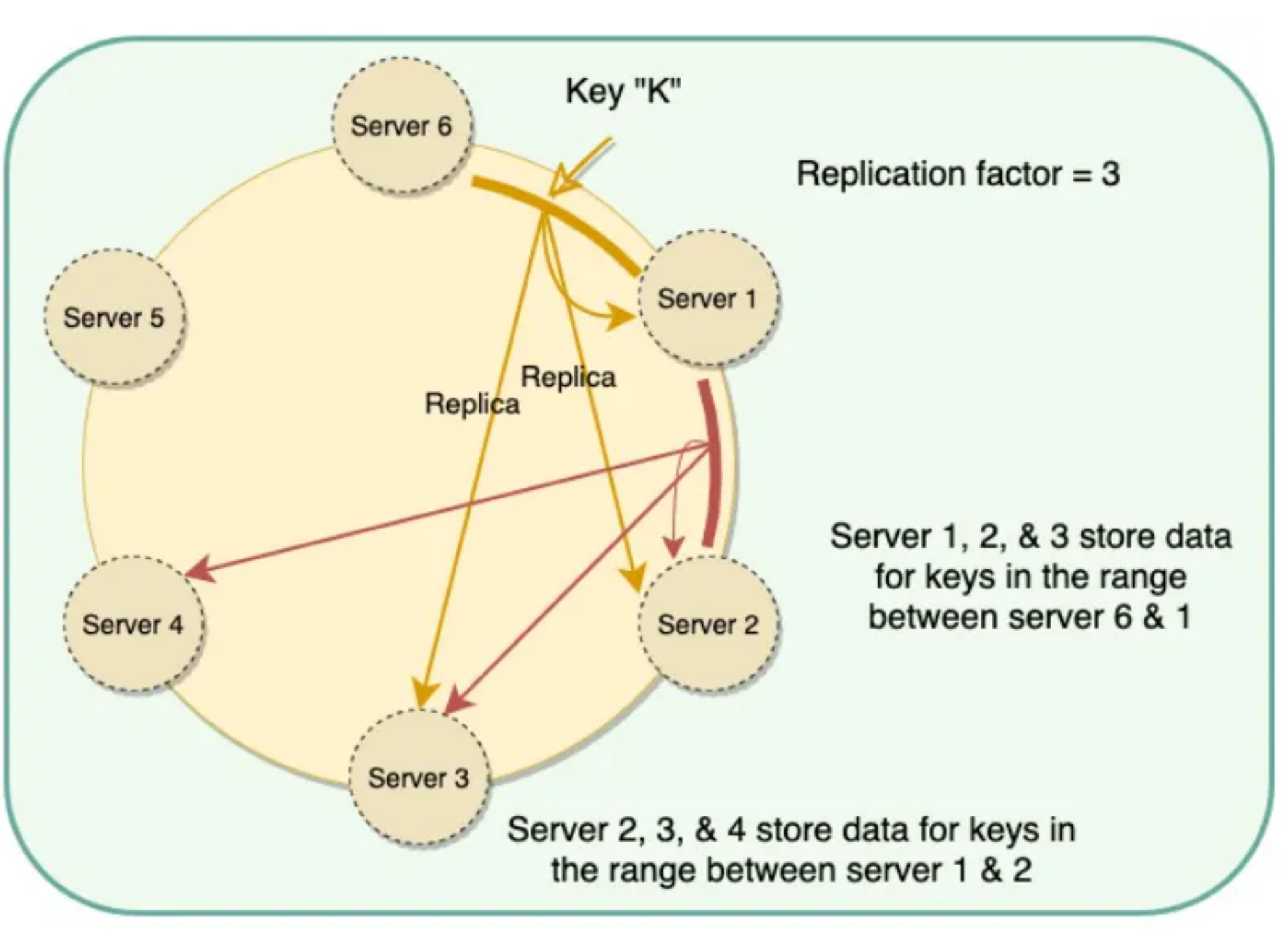
Data replication using Consistent Hashing

To ensure highly availability and durability, Consistent Hashing replicates each data item on multiple N nodes in the system where the value N is equivalent to the replication factor.

> The replication factor is the number of nodes that will receive the copy of the same data. For example, a replication factor of two means there are two copies of each data item, where each copy is stored on a different node.

Each key is assigned to a **coordinator node** (generally the first node that falls in the hash range), which first stores the data locally and then replicates it to $N-1$ clockwise successor nodes on the ring. This results in each node owning the region on the ring between it and its Nth predecessor. In an **eventually consistent** system, this replication is done asynchronously (in the background).

> In eventually consistent systems, copies of data don't always have to be identical as long as they are designed to eventually become consistent. In distributed systems, eventual consistency is used to achieve high availability.


Server 6
Key "K"
Replication factor = 3
Server 5
Server 1
Replica
Replica
Server 4
Server 2
Server 1, 2, & 3 store data for keys in the range between server 6 & 1
Server 3
Server 2, 3, & 4 store data for keys in the range between server 1 & 2

Replication in Consistent Hashing

Consistent Hashing in System Design Interviews

As we saw above, Consistent Hashing helps with efficiently partitioning and replicating data; therefore, any distributed system that needs to scale up or down or wants to achieve high availability through data replication can utilize Consistent Hashing. A few such examples could be:

- Any system working with a set of storage (or database) servers and needs to scale up or down based on the usage, e.g., the system could need more storage during Christmas because of high traffic.
- Any distributed system that needs dynamic adjustment of its cache usage by adding or removing cache servers based on the traffic load.
- Any system that wants to replicate its data shards to achieve high availability.

Consistent Hashing use cases

Amazon's Dynamo and Apache Cassandra use Consistent Hashing to distribute and replicate data across nodes.